

Comprehensive Matrix and Linear Algebra Overview

Levan Tsinadze

December 21, 2024

Outline (Slide 1)

1) Matrix Definition

2) Matrix Dimensions

- ▶ NumPy examples
- ▶ 2×2 , 3×3 , 2×3 , 3×2 matrices
- ▶ Visual examples
- ▶ $n \times m$ matrices
- ▶ Square matrices
- ▶ NumPy examples

3) Operations on Matrices

- ▶ Sum (2×3 example)
- ▶ Subtraction (3×2 example)
- ▶ Scalar multiplication (4×3 example)

4) Identity Matrix

- ▶ Definition
- ▶ 4×4 example
- ▶ $n \times n$ examples

Outline (Slide 2)

5) Matrix Multiplication

- ▶ $n \times m$ to $m \times k$ multiplication
- ▶ Visual examples
- ▶ Matrix-vector multiplication
($n \times m$ on $m \times 1$)
- ▶ 6×4 on 4×1
- ▶ 3×3 example
- ▶ NumPy (dot, @)
- ▶ Identity multiplication

6) Transpose Matrix

- ▶ $n \times m$ matrices
- ▶ 4×3 example
- ▶ NumPy examples

7) Inverse Matrix

- ▶ Determinant
- ▶ 2×2 , 3×3 , $n \times m$ examples
- ▶ Inverse matrix
- ▶ Visual examples
- ▶ NumPy examples

Outline (Slide 3)

8) Matrix Algebra

- ▶ NumPy examples

9) 2D and 3D Matrices

- ▶ NumPy examples

10) Tensors as Multidimensional Matrices

- ▶ NumPy examples
- ▶ Image as matrix
- ▶ Text as matrix

11) Linear Space and Basis Vectors

- ▶ 2D and 3D space examples
- ▶ Visual examples
- ▶ Linear independence
- ▶ n -dimensional spaces
- ▶ n -dimensional basis vectors
- ▶ Visual Example
- ▶ NumPy example

Outline (Slide 4)

12) Linear Transformation

- ▶ Transformation matrix from basis vectors with examples
- ▶ Proof of transformation matrix representation
- ▶ NumPy example
- ▶ Composition of linear transformation ($2 \times 3 \rightarrow 3 \times 4$)
- ▶ Visual examples

13) Affine Transformation

- ▶ NumPy example
- ▶ Composition of affine transformation ($2 \times 3 \rightarrow 3 \times 4$)
- ▶ Visual examples

14) Affine Transformation from $\mathbb{R}^n$ to $\mathbb{R}^1$

- ▶ Transformation formulas
- ▶ Affine transformation with sigmoid function to $\mathbb{R}^1$
- ▶ Visual examples
- ▶ NumPy examples

15) Affine Transformation and ReLU Function

- ▶ Visual examples
- ▶ NumPy examples

Matrix Definition

- ▶ A **matrix** is a rectangular array of numbers organized in rows and columns.
- ▶ Example of a 2×3 matrix:

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}.$$

Matrix Dimensions in NumPy

```
import numpy as np

A = np.array([[1, 2, 3],
              [4, 5, 6]])
print(A.shape)  # Output: (2, 3)
```

Examples of Various Dimensions

$$2 \times 2: \begin{pmatrix} a & b \\ c & d \end{pmatrix},$$

$$3 \times 3: \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix},$$

$$2 \times 3: \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix},$$

$$3 \times 2: \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix}.$$

Visual Matrix Examples (TikZ)

1	2	3
4	5	6

1	2
3	4
5	6

$n \times m$ Matrices

General form:

$$A = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1m} \\ a_{21} & a_{22} & \dots & a_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nm} \end{pmatrix}.$$

Square Matrices

- ▶ A matrix is **square** if it has the same number of rows and columns ($n \times n$).
- ▶ Examples:

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

NumPy: Various Matrix Shapes

```
import numpy as np
```

```
A_2x2 = np.array([[1,2],  
                  [3,4]])  
print(A_2x2.shape)  # Output: (2, 2)
```

```
A_3x3 = np.array([[1,2,3],  
                  [4,5,6],  
                  [7,8,9]])  
print(A_3x3.shape)  # Output: (3, 3)
```

```
A_2x3 = np.array([[1,2,3],  
                  [4,5,6]])  
print(A_2x3.shape)  # Output: (2, 3)
```

```
A_3x2 = np.array([[1,2],  
                  [3,4],  
                  [5,6]])
```

Sum of Matrices (2×3 Example)

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} + \begin{pmatrix} 7 & 8 & 9 \\ 0 & 1 & 2 \end{pmatrix} = \begin{pmatrix} 8 & 10 & 12 \\ 4 & 6 & 8 \end{pmatrix}.$$

Subtraction of Matrices (3×2 Example)

$$\begin{pmatrix} 4 & 5 \\ 2 & 3 \\ 1 & 1 \end{pmatrix} - \begin{pmatrix} 1 & 1 \\ 2 & 2 \\ 0 & 4 \end{pmatrix} = \begin{pmatrix} 3 & 4 \\ 0 & 1 \\ 1 & -3 \end{pmatrix}.$$

Scalar Multiplication and Distributive Law (4×3 Example)

- ▶ Multiply each entry by a scalar α .
- ▶ **Distributive Law:** $\alpha(A + B) = \alpha A + \alpha B$.

$$2 \cdot \begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \\ j & k & l \end{pmatrix} = \begin{pmatrix} 2a & 2b & 2c \\ 2d & 2e & 2f \\ 2g & 2h & 2i \\ 2j & 2k & 2l \end{pmatrix}.$$

Diagonal Matrix

- ▶ A **diagonal matrix** is a square matrix in which the entries outside the main diagonal are all zero.
- ▶ Formally, a matrix A is diagonal if:

$$A = \begin{pmatrix} a_{11} & 0 & \dots & 0 \\ 0 & a_{22} & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & a_{nn} \end{pmatrix}.$$

- ▶ **Example of a 3×3 Diagonal Matrix**:

$$D = \begin{pmatrix} 5 & 0 & 0 \\ 0 & -3 & 0 \\ 0 & 0 & 2 \end{pmatrix}.$$

- ▶ **Properties**:
 - ▶ **Multiplicative**: $D_1 \times D_2$ is also a diagonal matrix.
 - ▶ **Inverse**: If all diagonal entries are non-zero, D is invertible, and D^{-1} is obtained by taking reciprocals of the diagonal entries.
 - ▶ **Transpose**: $D^T = D$ (since it's symmetric).

Visual Example of a Diagonal Matrix

5	0	0
0	-3	0
0	0	2

Identity Matrix

- ▶ The identity matrix I_n is an $n \times n$ matrix with 1's on the main diagonal and 0's elsewhere.
- ▶ Example of a 4×4 identity matrix:

$$I_4 = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

- ▶ For any $n \times n$ matrix A , $I_n A = A I_n = A$.

$(n \times m)$ Times $(m \times k)$

$$(A_{n \times m}) \times (B_{m \times k}) = C_{n \times k}.$$

- ▶ The inner dimensions (m) must match.
- ▶ The result has outer dimensions $n \times k$.

Visual Example of Multiplication

$$\begin{array}{c} A_{2 \times 3} \\ \begin{array}{|c|c|c|} \hline a & b & c \\ \hline d & e & f \\ \hline \end{array} \end{array} \times \begin{array}{c} B_{3 \times 2} \\ \begin{array}{|c|c|} \hline p & q \\ \hline r & s \\ \hline t & u \\ \hline \end{array} \end{array} = \begin{array}{c} C_{2 \times 2} \\ \begin{array}{|c|c|} \hline A_{11} & A_{12} \\ \hline A_{21} & A_{22} \\ \hline \end{array} \end{array}$$

Vector-Matrix Multiplication Formula

Definition:

Let:

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1m} \\ a_{21} & a_{22} & \cdots & a_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nm} \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix}$$

Vector-Matrix Multiplication Formula

Definition:

Let:

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1m} \\ a_{21} & a_{22} & \cdots & a_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nm} \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix}$$

The product is:

$$A\mathbf{x} = \begin{bmatrix} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1m}x_m \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2m}x_m \\ \vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nm}x_m \end{bmatrix}.$$

Vector-Matrix Multiplication Formula

Definition:

Let:

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1m} \\ a_{21} & a_{22} & \cdots & a_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nm} \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix}$$

The product is:

$$A\mathbf{x} = \begin{bmatrix} a_{11}x_1 + a_{12}x_2 + \dots + a_{1m}x_m \\ a_{21}x_1 + a_{22}x_2 + \dots + a_{2m}x_m \\ \vdots \\ a_{n1}x_1 + a_{n2}x_2 + \dots + a_{nm}x_m \end{bmatrix}.$$

Key Insight: Each row in the result is a **dot product** between a row of A and the vector $\mathbf{x}$.

Vector-Matrix Multiplication as Linear Combination

Definition: The multiplication of a vector $\mathbf{x}$ with a matrix A can be interpreted as a **linear combination** of the **columns** of A .

Vector-Matrix Multiplication as Linear Combination

Definition: The multiplication of a vector $\mathbf{x}$ with a matrix A can be interpreted as a **linear combination** of the **columns** of A .

Mathematical Representation: Given:

$$A = [\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_n] \quad (\text{columns of } A)$$

and

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

The product is:

$$A\mathbf{x} = x_1\mathbf{a}_1 + x_2\mathbf{a}_2 + \dots + x_n\mathbf{a}_n$$

Vector-Matrix Multiplication as Linear Combination

Definition: The multiplication of a vector $\mathbf{x}$ with a matrix A can be interpreted as a **linear combination** of the **columns** of A .

Mathematical Representation: Given:

$$A = [\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_n] \quad (\text{columns of } A)$$

and

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

The product is:

$$A\mathbf{x} = x_1\mathbf{a}_1 + x_2\mathbf{a}_2 + \dots + x_n\mathbf{a}_n$$

Key Insight: Each coordinate x_i scales the corresponding column vector $\mathbf{a}_i$.

Example of Vector-Matrix Multiplication

Given:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} 5 \\ 6 \end{bmatrix}$$

Example of Vector-Matrix Multiplication

Given:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} 5 \\ 6 \end{bmatrix}$$

Compute:

$$A\mathbf{x} = 5 \begin{bmatrix} 1 \\ 3 \end{bmatrix} + 6 \begin{bmatrix} 2 \\ 4 \end{bmatrix}$$

Example of Vector-Matrix Multiplication

Given:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} 5 \\ 6 \end{bmatrix}$$

Compute:

$$A\mathbf{x} = 5 \begin{bmatrix} 1 \\ 3 \end{bmatrix} + 6 \begin{bmatrix} 2 \\ 4 \end{bmatrix}$$

Result:

$$A\mathbf{x} = \begin{bmatrix} 5 + 12 \\ 15 + 24 \end{bmatrix} = \begin{bmatrix} 17 \\ 39 \end{bmatrix}$$

Matrix-Vector Multiplication ($n \times m$ on $m \times 1$)

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 1 \cdot x_1 + 2 \cdot x_2 + 3 \cdot x_3 \\ 4 \cdot x_1 + 5 \cdot x_2 + 6 \cdot x_3 \end{pmatrix}.$$

6×4 on 4×1 Vector Example

$$A_{6 \times 4} \cdot \vec{x}_{4 \times 1} = \vec{y}_{6 \times 1}.$$

- ▶ Example matrix and vector:

$$A = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \\ 17 & 18 & 19 & 20 \\ 21 & 22 & 23 & 24 \end{pmatrix}, \quad \vec{x} = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix}.$$

- ▶ The result $\vec{y}$ is a 6×1 vector.

3×3 Multiplication Example

$$\begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix} \times \begin{pmatrix} p & q & r \\ s & t & u \\ v & w & x \end{pmatrix} =$$

$$\begin{pmatrix} ap + bs + cv & aq + bt + cw & ar + bu + cx \\ dp + es + fv & dq + et + fw & dr + eu + fx \\ gp + hs + iv & gq + ht + iw & gr + hu + ix \end{pmatrix}$$

NumPy Matrix Multiplication

```
import numpy as np

A = np.array([[1,2,3],
              [4,5,6]])
B = np.array([[7,8],
              [9,10],
              [11,12]])

C1 = A.dot(B)
C2 = A @ B # Python 3.5+ operator
print("C1:\n", C1)
print("C2:\n", C2)
```

Multiplication with Identity

$$I_n A = A I_n = A \quad (\text{if } A \text{ is } n \times n).$$

- ▶ Multiplying any square matrix by the identity matrix leaves it unchanged.

Transpose

$$(A^T)_{ij} = A_{ji}.$$

- ▶ Transposes an $n \times m$ matrix into an $m \times n$ matrix.

4×3 Transpose Example

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 0 & 1 & 2 \end{pmatrix} \Rightarrow A^T = \begin{pmatrix} 1 & 4 & 7 & 0 \\ 2 & 5 & 8 & 1 \\ 3 & 6 & 9 & 2 \end{pmatrix}.$$

Transpose in NumPy

```
import numpy as np

A = np.array([[1,2,3],
              [4,5,6],
              [7,8,9],
              [0,1,2]])

A_T = A.T
print("A Transposed:\n", A_T)
```

Matrix Determinant

- ▶ For a square matrix, the determinant $\det(A)$ determines if A is invertible.
- ▶ A is invertible if $\det(A) \neq 0$.

2×2 Determinant and Inverse

$$\det \begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc.$$

$$\text{If } ad - bc \neq 0, \text{ then } A^{-1} = \frac{1}{ad - bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}.$$

3×3 Determinant

$$\det \begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix} = a(ei - fh) - b(di - fg) + c(dh - eg).$$

$n \times m$ Matrices

- ▶ Non-square matrices ($n \neq m$) do not have a standard inverse.
- ▶ Only square matrices ($n = m$) can potentially be invertible.

Inverse Matrix

$$A^{-1}A = AA^{-1} = I_n \quad (\text{if } A \text{ is } n \times n \text{ and non-singular}).$$

- Example for 2×2 matrix:

$$A^{-1} = \frac{1}{ad - bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}.$$

Inverse Matrix Visual (2×2)

$$\begin{array}{c} A \\ \begin{array}{|c|c|} \hline a & b \\ \hline c & d \\ \hline \end{array} \end{array} = \begin{array}{c} A^{-1} \\ \begin{array}{|c|c|} \hline \frac{d}{ad-bc} & \frac{-b}{ad-bc} \\ \hline \frac{-c}{ad-bc} & \frac{a}{ad-bc} \\ \hline \end{array} \end{array}$$

Inverse in NumPy

```
import numpy as np

A = np.array([[1.,2.],
              [3.,4.]])
A_inv = np.linalg.inv(A)
print("A Inverse:\n", A_inv)
```

Matrix Algebra in NumPy

```
import numpy as np

# Define matrices
A = np.random.randn(3,3)
B = np.random.randn(3,3)

# Operations
C = A + B          # Sum
D = A - B          # Subtraction
E = A.dot(B)       # Multiplication
F = np.linalg.inv(A) # Inverse (if A is invertible)

print("Sum:\n", C)
print("Subtraction:\n", D)
print("Multiplication:\n", E)
print("Inverse of A:\n", F)
```

2D and 3D Matrices in NumPy

```
import numpy as np

# 2D array
A_2d = np.array([[1,2,3],
                 [4,5,6]])
print("2D shape:", A_2d.shape)  # Output: (2, 3)

# 3D array (e.g., RGB image)
A_3d = np.random.randn(2,3,4)  # Shape: (2,3,4)
print("3D shape:", A_3d.shape)
```

Tensors in NumPy

```
import numpy as np

# 3D tensor
T = np.random.randn(3,3,3)
print("Tensor shape:", T.shape)  # Output: (3,3,3)

# Higher-dimensional tensor
T4 = np.random.randn(2,3,4,5)
print("4D Tensor shape:", T4.shape)
```

Image as Matrix

- ▶ **Grayscale Image:** 2D matrix of pixel intensities.
- ▶ **Color Image (RGB):** 3D tensor (height $\times$ width $\times$ channels).

Text as Matrix

- ▶ **Embedding:** Each word is represented as a vector, forming a matrix (sequence length $\times$ embedding dimension).
- ▶ Used in Natural Language Processing (NLP) for various applications.

Linear Independence and Dimension

- ▶ Vectors $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k$ are **linearly independent** if:

$$\alpha_1 \mathbf{v}_1 + \dots + \alpha_k \mathbf{v}_k = \mathbf{0} \implies \alpha_1 = \dots = \alpha_k = 0.$$

- ▶ A basis for an n -dimensional space has exactly n linearly independent vectors.

Basis Vectors - Definition

Definition: Basis vectors are a set of vectors in a vector space that are:

- ▶ **Linearly Independent.**
- ▶ **Span the Vector Space.**

Standard Basis Vectors in $\mathbb{R}^n$:

- ▶ In $\mathbb{R}^2$: $e_1 = [1, 0]^T$, $e_2 = [0, 1]^T$.
- ▶ In $\mathbb{R}^3$: $e_1 = [1, 0, 0]^T$, $e_2 = [0, 1, 0]^T$, $e_3 = [0, 0, 1]^T$.

Basis Vectors - Definition

Definition: Basis vectors are a set of vectors in a vector space that are:

- ▶ **Linearly Independent.**
- ▶ **Span the Vector Space.**

Standard Basis Vectors in $\mathbb{R}^n$:

- ▶ In $\mathbb{R}^2$: $e_1 = [1, 0]^T$, $e_2 = [0, 1]^T$.
- ▶ In $\mathbb{R}^3$: $e_1 = [1, 0, 0]^T$, $e_2 = [0, 1, 0]^T$, $e_3 = [0, 0, 1]^T$.

Properties:

- ▶ Any vector can be written as a linear combination of basis vectors.
- ▶ Basis vectors define coordinate axes.

Linear Combinations

Definition: A vector $\mathbf{v}$ can be written as:

$$\mathbf{v} = c_1 \mathbf{e}_1 + c_2 \mathbf{e}_2 + \cdots + c_n \mathbf{e}_n$$

Examples:

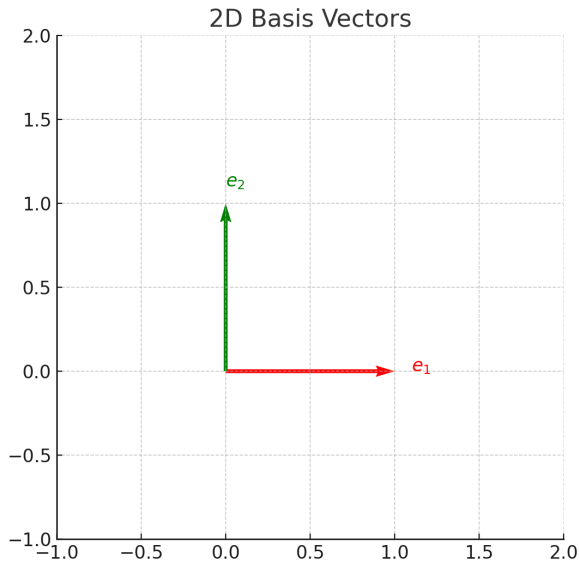
► In $\mathbb{R}^2$:

$$\mathbf{u} = [3, 4] = 3[1, 0] + 4[0, 1]$$

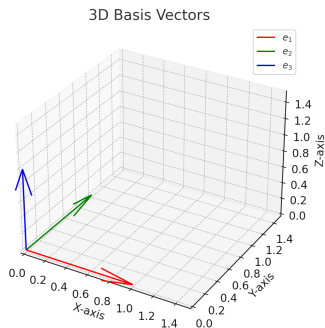
► In $\mathbb{R}^3$:

$$\mathbf{v} = [1, 2, 3] = 1[1, 0, 0] + 2[0, 1, 0] + 3[0, 0, 1]$$

Visual Example: Basis Vectors in 2D



Visual Example: Basis Vectors in 3D



Example: Decomposing vector $\mathbf{v} = [1, 2, 3]$ into basis vectors in 3D.

Exercises

Task:

- Express the following vectors as linear combinations of basis vectors:
 1. $\mathbf{w} = [2, -1]$ in $\mathbb{R}^2$.
 2. $\mathbf{x} = [1, 0, -2]$ in $\mathbb{R}^3$.

Exercises

Task:

- ▶ Express the following vectors as linear combinations of basis vectors:
 1. $\mathbf{w} = [2, -1]$ in $\mathbb{R}^2$.
 2. $\mathbf{x} = [1, 0, -2]$ in $\mathbb{R}^3$.
- ▶ Verify your answers by reconstructing the vectors.

Key Takeaways

Key Concepts:

- ▶ Basis vectors define the axes in vector spaces.
- ▶ Any vector can be represented as a linear combination of basis vectors.
- ▶ Visualization helps in understanding the decomposition into components.

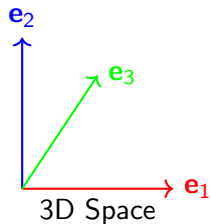
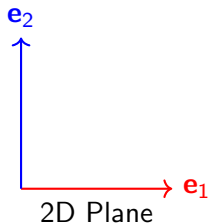
Next Steps: Apply these concepts to higher-dimensional spaces and affine transformations.

2D and 3D Space Examples

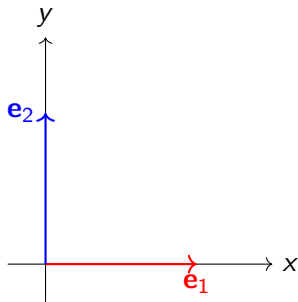
$$\mathbf{e}_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \mathbf{e}_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \quad (2D)$$

$$\mathbf{e}_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \mathbf{e}_2 = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \mathbf{e}_3 = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \quad (3D)$$

Visual Example of Basis Vectors



Basis in 2D Space



Checking Linear Independence in NumPy

```
import numpy as np

v1 = np.array([1,2,3])
v2 = np.array([2,4,6])
# v2 is multiple of v1 => dependent

M = np.column_stack((v1, v2))
rank = np.linalg.matrix_rank(M)
print("Rank =", rank)  # Output: Rank = 1
```

Matrix from Basis Vectors

- ▶ If $T(\mathbf{e}_i)$ = the i -th column of A , then for any $\mathbf{x}$,

$$T(\mathbf{x}) = A\mathbf{x}.$$

Proof Sketch

- ▶ A linear map $T : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is fully determined by $T(\mathbf{e}_1), \dots, T(\mathbf{e}_n)$.
- ▶ Each $T(\mathbf{e}_i)$ becomes the i -th column of matrix A representing T .

Linear Mapping of Basis Vectors

Linear Mapping Property: A linear mapping $T : \mathbb{R}^n \rightarrow \mathbb{R}^m$ satisfies:

- ▶ $T(\mathbf{u} + \mathbf{v}) = T(\mathbf{u}) + T(\mathbf{v})$ (Additivity).
- ▶ $T(c\mathbf{u}) = cT(\mathbf{u})$ (Homogeneity).

Linear Mapping of Basis Vectors

Linear Mapping Property: A linear mapping $T : \mathbb{R}^n \rightarrow \mathbb{R}^m$ satisfies:

- ▶ $T(\mathbf{u} + \mathbf{v}) = T(\mathbf{u}) + T(\mathbf{v})$ (Additivity).
- ▶ $T(c\mathbf{u}) = cT(\mathbf{u})$ (Homogeneity).

Mapping Basis Vectors:

- ▶ Let T be a linear transformation and $\mathbf{e}_i$ are basis vectors.
- ▶ The mapping of any vector $\mathbf{v}$ is:

$$T(\mathbf{v}) = T(c_1\mathbf{e}_1 + c_2\mathbf{e}_2 + \dots + c_n\mathbf{e}_n)$$

$$T(\mathbf{v}) = c_1 T(\mathbf{e}_1) + c_2 T(\mathbf{e}_2) + \dots + c_n T(\mathbf{e}_n)$$

Linear Mapping of Basis Vectors

Linear Mapping Property: A linear mapping $T : \mathbb{R}^n \rightarrow \mathbb{R}^m$ satisfies:

- ▶ $T(\mathbf{u} + \mathbf{v}) = T(\mathbf{u}) + T(\mathbf{v})$ (Additivity).
- ▶ $T(c\mathbf{u}) = cT(\mathbf{u})$ (Homogeneity).

Mapping Basis Vectors:

- ▶ Let T be a linear transformation and $\mathbf{e}_i$ are basis vectors.
- ▶ The mapping of any vector $\mathbf{v}$ is:

$$T(\mathbf{v}) = T(c_1\mathbf{e}_1 + c_2\mathbf{e}_2 + \dots + c_n\mathbf{e}_n)$$

$$T(\mathbf{v}) = c_1 T(\mathbf{e}_1) + c_2 T(\mathbf{e}_2) + \dots + c_n T(\mathbf{e}_n)$$

Key Insight: Linear mappings preserve linear combinations.

Linear Transformation in NumPy

```
import numpy as np

A = np.array([[2,1],
              [0,3]])
x = np.array([1,4])

Tx = A @ x
print("T(x) =", Tx)  # Output: T(x) = [6 12]
```

Composition of Linear Transformations

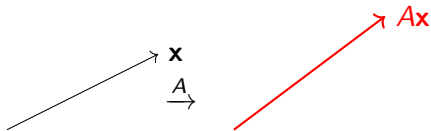
$$T_2 \circ T_1(\mathbf{x}) = A_2(A_1\mathbf{x}) = (A_2A_1)\mathbf{x}.$$

► Example:

$$A_1 \text{ is } 2 \times 3, \quad A_2 \text{ is } 3 \times 4 \quad \Rightarrow \quad A_2A_1 \text{ is } 2 \times 4.$$

► Ensures dimensional compatibility for matrix multiplication.

Visual Example of $T(\mathbf{x})$



Introduction to Linear Transformations

- ▶ A **linear transformation** is a mapping $T : \mathbb{R}^n \rightarrow \mathbb{R}^m$ that satisfies:
 1. **Additivity**: $T(\mathbf{u} + \mathbf{v}) = T(\mathbf{u}) + T(\mathbf{v})$
 2. **Homogeneity**: $T(c\mathbf{u}) = cT(\mathbf{u})$
- ▶ Every linear transformation can be represented by a matrix.
- ▶ Understanding how to generate these matrices is crucial for applications in computer graphics, data science, and more.

Generating Transformation Matrices

- ▶ To generate a linear transformation matrix A , determine how the transformation T maps each of the standard basis vectors $\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_n$.
- ▶ The images $T(\mathbf{e}_1), T(\mathbf{e}_2), \dots, T(\mathbf{e}_n)$ become the ****columns**** of matrix A .
- ▶ Mathematically:

$$A = \left(\begin{array}{c|c|c|c} | & | & & | \\ T(\mathbf{e}_1) & T(\mathbf{e}_2) & \dots & T(\mathbf{e}_n) \\ | & | & & | \end{array} \right) = [T(\mathbf{e}_1) \ T(\mathbf{e}_2) \ \dots \ T(\mathbf{e}_n)]$$

- ▶ This ensures that for any vector $\mathbf{x} \in \mathbb{R}^n$,

$$T(\mathbf{x}) = A\mathbf{x}.$$

Scaling Transformation

- ▶ **Definition**: Scaling stretches or shrinks vectors by a constant factor along each axis.
- ▶ **Scaling Factors**: s_x along the x-axis and s_y along the y-axis.

$$A = \begin{pmatrix} s_x & 0 \\ 0 & s_y \end{pmatrix}$$

- ▶ **Example**: Scale by 2 along x-axis and 3 along y-axis.

$$A = \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix}$$

Rotation Transformation

- ▶ **Definition**: Rotates vectors by an angle θ counterclockwise around the origin.
- ▶ **Rotation Matrix**:

$$A = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$$

- ▶ **Example**: Rotate by 45 degrees ($\theta = \frac{\pi}{4}$).

$$A = \begin{pmatrix} \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \end{pmatrix}$$

Shearing Transformation

- ▶ **Definition**: Shearing shifts each point in a fixed direction, proportional to its distance from a line that is parallel to that direction.
- ▶ **Types of Shear**:
 1. **Horizontal Shear**: Shifts x-coordinate based on y.
 2. **Vertical Shear**: Shifts y-coordinate based on x.
- ▶ **Shear Matrices**:

Horizontal Shear: $A = \begin{pmatrix} 1 & k \\ 0 & 1 \end{pmatrix}$, Vertical Shear: $A = \begin{pmatrix} 1 & 0 \\ k & 1 \end{pmatrix}$

Example: Horizontal shear with $k = 1.5$

$$A = \begin{pmatrix} 1 & 1.5 \\ 0 & 1 \end{pmatrix}$$

Reflection Transformation

- ▶ ****Definition****: Reflects vectors across a specified axis or line.
- ▶ ****Reflection Across the x-axis****:

$$A = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

- ▶ ****Reflection Across the y-axis****:

$$A = \begin{pmatrix} -1 & 0 \\ 0 & 1 \end{pmatrix}$$

- ▶ ****Reflection Across the Line $y = x$ ****:

$$A = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

Scaling Transformation in NumPy

- Use NumPy to create a scaling transformation matrix.

1. Scaling Transformation:

```
1 import numpy as np
2
3 # Scaling Transformation
4 def scaling_matrix(sx, sy):
5     return np.array([[sx, 0],
6                     [0, sy]])
7
8 # Example usage
9 sx, sy = 2, 3
10 A_scale = scaling_matrix(sx, sy)
11 print("Scaling Matrix:\n", A_scale)
```

Rotation Transformation in NumPy

- Use NumPy to create a rotation transformation matrix.

2. Rotation Transformation:

```
1 import numpy as np
2
3 # Rotation Transformation
4 def rotation_matrix(theta):
5     return np.array([[np.cos(theta), -np.sin(
6         theta)],
7                       [np.sin(theta),  np.cos(
8         theta)]]])
9
10 # Example usage
11 theta = np.pi / 4 # 45 degrees
12 A_rotate = rotation_matrix(theta)
13 print("Rotation Matrix (45 degrees):\n",
14       A_rotate)
```

Shearing and Reflection Matrices in NumPy

- Use NumPy to create transformation matrices for shearing and reflection.

3. Shearing Transformation:

```
1 # Shearing Transformation (Horizontal)
2 def shear_horizontal(k):
3     return np.array([[1, k],
4                       [0, 1]])
5
6 # Example usage
7 k = 1.5
8 A_shear = shear_horizontal(k)
9 print("Horizontal Shear Matrix:\n", A_shear)
```

Shearing and Reflection Matrices in NumPy

- Use NumPy to create transformation matrices for shearing and reflection.

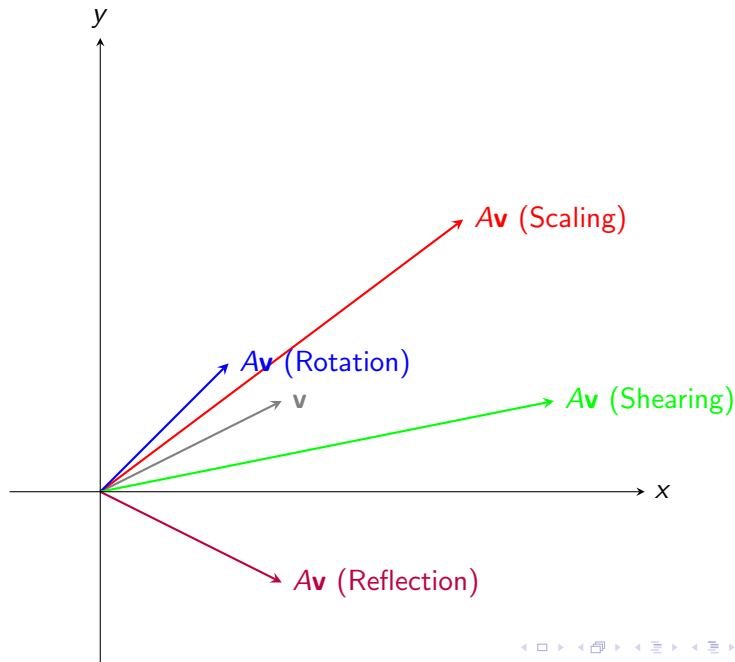
3. Shearing Transformation:

```
1 # Shearing Transformation (Horizontal)
2 def shear_horizontal(k):
3     return np.array([[1, k],
4                      [0, 1]])
5
6 # Example usage
7 k = 1.5
8 A_shear = shear_horizontal(k)
9 print("Horizontal Shear Matrix:\n", A_shear)
```

4. Reflection Transformation:

```
1 # Reflection Transformation (across x-axis)
2 def reflection_x():
3     return np.array([[1, 0],
4                      [0, -1]])
5
```


Visual Examples of Linear Transformations



Affine Transformation between $\mathbb{R}^m$ and $\mathbb{R}^n$

- ▶ An **affine transformation** is a function $T : \mathbb{R}^m \rightarrow \mathbb{R}^n$ that can be expressed as:

$$T(\mathbf{x}) = A\mathbf{x} + \mathbf{b}$$

- ▶ A is an $n \times m$ **transformation matrix**.
 - ▶ $\mathbf{b}$ is a fixed vector in $\mathbb{R}^n$ known as the **translation vector**.
- ▶ Affine transformations combine **linear transformations** with **translations**.
- ▶ Unlike linear transformations, affine transformations do not necessarily preserve the origin unless $\mathbf{b} = \mathbf{0}$.

Properties of Affine Transformations

- ▶ ****Combination of Operations****:
 - ▶ Affine transformations are combinations of linear transformations and translations.
 - ▶ Formally, $T(\mathbf{x}) = A\mathbf{x} + \mathbf{b}$ where A is linear and $\mathbf{b}$ is a translation.
- ▶ ****Closure****:
 - ▶ The set of affine transformations is closed under composition.
- ▶ ****Preservation of Parallelism****:
 - ▶ Affine transformations preserve parallel lines.
- ▶ ****Linear Transformation as a Special Case****:
 - ▶ When $\mathbf{b} = \mathbf{0}$, the affine transformation reduces to a linear transformation.

Affine Transformation

```
import numpy as np

A = np.array([[2,1],[0,3]])
b = np.array([1, -1])
x = np.array([3,2])

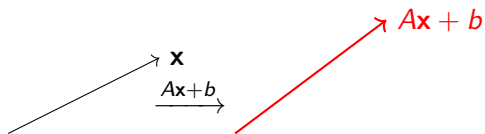
y = A @ x + b
print("Affine transform =", y)  # Output: Affine transform
```

Composition of Affine Transformations

- ▶ $f_1(\mathbf{x}) = A_1\mathbf{x} + b_1$
- ▶ $f_2(\mathbf{y}) = A_2\mathbf{y} + b_2$
- ▶ Composition:

$$f_2 \circ f_1(\mathbf{x}) = A_2(A_1\mathbf{x} + b_1) + b_2 = A_2A_1\mathbf{x} + (A_2b_1 + b_2).$$

Affine Transformation Visual



Transformation Formulas

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b, \quad \mathbf{w} \in \mathbb{R}^n, \quad b \in \mathbb{R}.$$

Affine Transformation with Sigmoid

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

w = np.array([1.5, -2.0])
b = 0.5
x = np.array([2.0, 1.0])

z = w @ x + b
y = sigmoid(z)
print("Affine + Sigmoid =", y)  # Output: Affine + Sigmoid
```


Visual: Affine $\rightarrow$ Sigmoid



NumPy Implementation

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

x = np.array([0.5, 2.0])
z = w @ x + b
output = sigmoid(z)
print("Output:", output)  # Output: Output: 0.8807970779778
```

Affine + ReLU (Visual)



ReLU in NumPy

```
import numpy as np

def relu(z):
    return np.maximum(0, z)

w = np.array([1., -1.])
b = -0.5
x = np.array([2., 3.])

z = w @ x + b
y = relu(z)
print("Affine + ReLU =", y)  # Output: Affine + ReLU = 1.5
```